Superstore Data Analytics

End-to-End Project Report

A comprehensive data analytics project covering data cleaning in Python, relational database design in PostgreSQL, analytical SQL queries, and exploratory data visualization — demonstrating the full lifecycle from raw data to actionable business insights.

Author	Hrushikesh
Dataset	Sample Superstore (Kaggle)
Tools	Python (Pandas, Matplotlib, Seaborn), PostgreSQL, Jupyter Notebook
Period	January 2014 – December 2017
Records	9,993 transaction line items across 5,009 orders

Table of Contents

1. Project Overview

2. Problem Statement

3. Data Pipeline & Architecture

4. Data Cleaning & Preparation (Python)

5. Database Design & SQL Implementation (PostgreSQL)

6. Data Visualization & Exploratory Analysis

7. Key Insights & Findings

8. Recommendations

9. Conclusion & Future Work

1. Project Overview

This project demonstrates an end-to-end data analytics workflow using the Superstore dataset — a widely used retail dataset containing four years of transaction data for a fictitious US-based office supplies retailer. The project spans the entire analytics lifecycle: ingesting raw data, cleaning and transforming it in Python, designing a normalized relational database in PostgreSQL, writing analytical SQL queries, and creating data visualizations that surface actionable business insights.

Objectives

- Clean and prepare raw transactional data for analysis using Python (Pandas).
- Design a normalized PostgreSQL schema (star schema with dimension and fact tables).
- Write comprehensive SQL analytics queries covering sales, products, customers, shipping, and geography.
- Build 14 exploratory visualizations in Python (Matplotlib/Seaborn) to communicate findings.
- Derive actionable business recommendations from the analysis.

Key Deliverables

- **Superstore_code.ipynb** — Jupyter notebook with data cleaning and EDA visualizations.
- **SUPERSTORE_SCHEMA.sql** — PostgreSQL database schema (5 tables, indexes, constraints).
- **SUPERSTORE_DATA_LOAD_SCRIPT.sql** — ETL script to load cleaned CSV into the database.
- **SUPERSTORE_ANALYTICS_QUERIES.sql** — 18 analytical SQL queries across 6 business domains.
- **Sample_Superstore_Cleaned.csv** — The cleaned, analysis-ready dataset.

$2.30M	$286K	12.5%	5,009	793
Total Sales	Total Profit	Profit Margin	Total Orders	Unique Customers

Figure 1: High-Level KPIs — Superstore Dataset (2014–2017)

2. Problem Statement

The Superstore retailer operates across 49 US states, serving three customer segments (Consumer, Corporate, Home Office) with products spanning Furniture, Office Supplies, and Technology. Despite generating over $2.3 million in revenue across four years, the company's overall profit margin sits at only 12.5%, and certain product lines and regions are actively losing money.

The business leadership needs answers to several critical questions:

- Which product categories and sub-categories are profitable, and which are eroding margins?
- What is the impact of discounting on profitability — and where is the threshold beyond which discounts destroy value?
- Which geographic regions and states are underperforming, and why?
- What does the customer base look like in terms of retention and repeat purchasing behaviour?
- How does shipping performance vary across modes and regions?
- What is the overall growth trajectory, and are there seasonal patterns the business can exploit?

This project aims to answer these questions through a structured, reproducible analytics pipeline — from raw data to insights — using industry-standard tools (Python, PostgreSQL) and best practices (normalized schema design, version-controlled SQL scripts, documented Jupyter notebooks).

3. Data Pipeline & Architecture

The project follows a structured pipeline that mirrors real-world data engineering workflows:

Stage	Tool	Description
1. Ingestion	Python / Pandas	Read raw CSV (9,994 rows, 21 columns) with latin-1 encoding
2. Cleaning	Python / Pandas	Type casting, whitespace trimming, deduplication, derived columns
3. Export	Python / Pandas	Export cleaned CSV (9,993 rows, 23 columns) for SQL ingestion
4. Schema Design	PostgreSQL	5-table normalized schema (star schema with 3 dimensions + 2 facts)
5. ETL / Load	PostgreSQL	Staging table → dimension loads → fact loads with validation
6. Analytics	PostgreSQL	18 analytical queries across 6 business domains
7. Visualization	Python / Matplotlib	14 charts with business insights, organized by SQL sections

Figure 2: Project Pipeline — From Raw Data to Insights

Database Schema Design

The PostgreSQL database uses a star schema with three dimension tables (geography, customers, products) and two fact tables (orders as the fact header, order_items as the fact detail). This design denormalizes the original flat CSV into a proper relational structure, eliminating data redundancy and enabling efficient analytical queries with JOINs.

Table	Type	Rows	Purpose
geography	Dimension	631	Postal code, city, state, region, country
customers	Dimension	793	Customer ID, name, segment
products	Dimension	1,862	Product ID, name, category, sub-category
orders	Fact (header)	5,009	Order ID, dates, ship mode, FK to customer & geo
order_items	Fact (detail)	9,993	Sales, quantity, discount, profit, derived flags

Figure 3: Database Schema — Table Summary

Strategic indexes were created on frequently filtered columns (order_date, customer_id, geo_id, product_id, is_loss, high_discount) and a composite index on (order_date, geo_id) for time-series queries by region.

4. Data Cleaning & Preparation

The raw dataset (Sample_Superstore.csv) required several cleaning steps before it was suitable for analysis. All cleaning was performed in a Jupyter notebook using Python and Pandas, ensuring reproducibility.

Cleaning Steps Performed

Step	Action	Detail
1	Whitespace trimming	Applied .str.strip() to all string columns to remove leading/trailing spaces
2	Date parsing	Converted Order Date and Ship Date from MM/DD/YYYY strings to datetime objects
3	Postal code fix	Zero-padded postal codes to 5 digits (e.g., '5401' → '05401') to preserve leading zeros
4	Column drop	Removed 'Row ID' — a non-informative sequential identifier
5	Text standardisation	Applied .str.title() to Ship Mode, Segment, Region, Category, Sub-Category
6	Derived: Days_to_Ship	Computed as (Ship Date – Order Date).days
7	Derived: Is Loss	Boolean flag: True where Profit < 0
8	Derived: High Discount	Boolean flag: True where Discount > 0.8
9	Deduplication	Checked and removed duplicate rows (1 duplicate found and removed)
10	Validation	Asserted: all quantities > 0, discounts in [0, 1], no negative shipping days, no nulls

Figure 4: Data Cleaning Steps

The cleaning process transformed the raw dataset from 9,994 rows with 21 columns to a clean dataset of 9,993 rows with 23 columns (original columns minus Row ID, plus three derived columns). The cleaned data was exported as Sample_Superstore_Cleaned.csv for downstream SQL ingestion.

5. SQL Implementation

The SQL layer consists of three scripts that together form a complete, reproducible database pipeline:

5.1 Schema Design (SUPERSTORE_SCHEMA.sql)

The schema script creates a 5-table star schema with primary keys, foreign key constraints, NOT NULL constraints, default values, and a UNIQUE constraint on geography (postal_code, city, state). It also creates 11 performance indexes covering the most common query patterns — date range filtering, JOIN columns, boolean flag filtering, and a composite index for time-series by region.

5.2 ETL / Data Load (SUPERSTORE_DATA_LOAD_SCRIPT.sql)

The load script follows a staging-table pattern common in production ETL pipelines. It creates a flat staging table mirroring the CSV, uses COPY to bulk-load the data, validates the staging data (null checks, date format checks), then loads each dimension table with deduplication (DISTINCT ON, ON CONFLICT DO NOTHING) before loading the fact tables. A post-load validation step confirms row counts and aggregate totals match between staging and production tables.

5.3 Analytics Queries (SUPERSTORE_ANALYTICS_QUERIES.sql)

The analytics script contains 18 queries organized into 6 sections. These queries demonstrate a range of SQL techniques — aggregation, multi-table JOINs, CASE expressions, CTEs, window functions (NTILE, LAG, RANK, running SUM), and FILTER clauses.

Section	Queries	Techniques Demonstrated
1. Sales Performance	Yearly KPIs, Monthly trends, By region, By segment	GROUP BY, EXTRACT, TO_CHAR, NULLIF
2. Product Analytics	Category margins, Top/bottom products, Discount impact	CASE expressions, FILTER clause, multi-level GROUP BY
3. Customer Analytics	Top customers, Repeat rate, RFM analysis	CTEs, NTILE window function, subqueries
4. Shipping	Avg days by mode, Performance by region	AVG, MIN, MAX aggregations with JOINs
5. Geographic	Top/bottom states by sales and profit	Multi-table JOINs, FILTER clause
6. Advanced	YTD running totals, MoM growth, Product rankings	Running SUM, LAG, RANK, PARTITION BY

Figure 5: SQL Analytics Queries — Summary by Section

Example: RFM Analysis Query (Section 3.3)

The RFM (Recency, Frequency, Monetary) query uses a CTE to calculate each customer's last order date, order count, and total spend, then applies NTILE(4) window functions to score each customer on a 1–4 scale across all three dimensions. This segmentation enables targeted marketing — e.g., customers with high monetary but low recency scores are at risk of churn and should be prioritised for re-engagement.

Example: Month-over-Month Growth (Section 6.2)

This query uses a CTE for monthly aggregation and the LAG window function to compute the previous month's sales, then calculates percentage growth with safe division (NULLIF to avoid divide-by-zero). This pattern is directly transferable to any time-series growth analysis.

6. Data Visualization & Exploratory Analysis

Fourteen visualizations were created using Matplotlib and Seaborn, organized to mirror the SQL analytics sections. Each chart was designed to answer a specific business question and includes a written insight interpreting the data.

6.1 Yearly Sales & Profit Performance

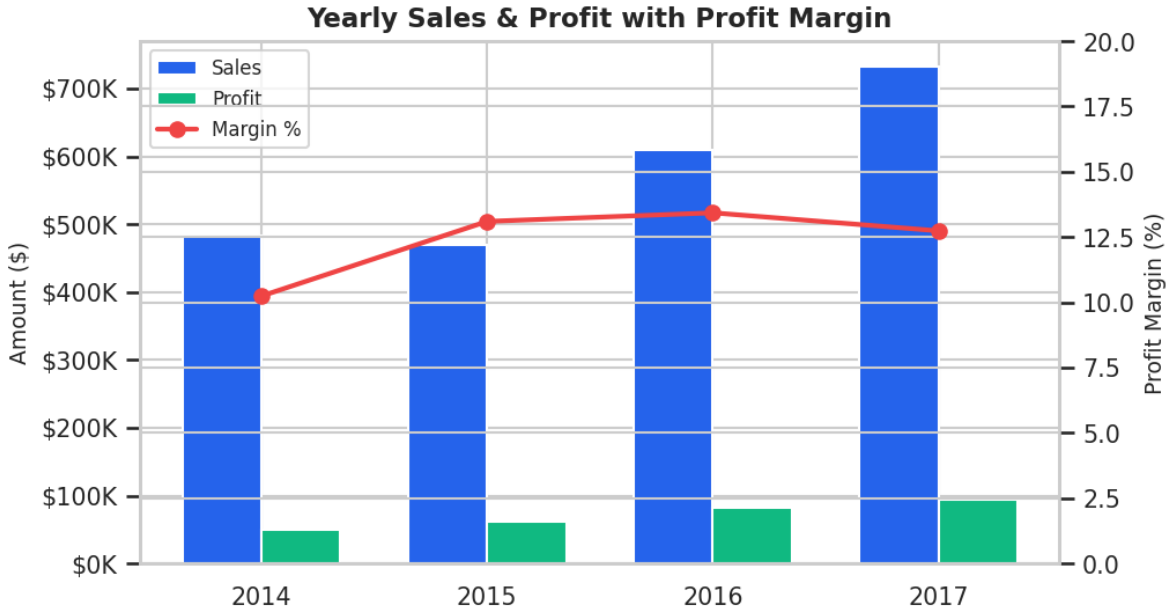

Figure 6: Yearly Sales & Profit with Profit Margin Overlay (2014–2017)

This grouped bar chart with a dual-axis margin line shows the company's growth trajectory. Sales grew from $484K in 2014 to $733K in 2017 — a 51% increase over four years. Profit margins fluctuated between 10.2% and 13.4%, suggesting that while revenue grew, margin improvement was inconsistent.

Key Insight: Revenue grew 51% over four years, but profit margin peaked in 2016 (13.4%) and slightly dipped in 2017 (12.7%), suggesting that growth came partly from margin-dilutive channels like heavy discounting.

6.2 Monthly Sales Trend

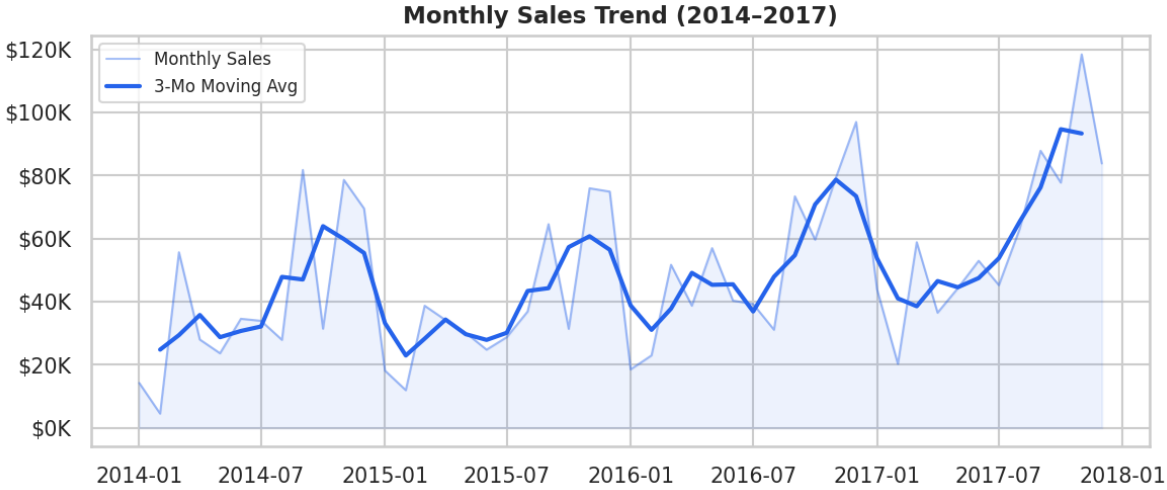

Figure 7: Monthly Sales with 3-Month Moving Average (2014–2017)

The time-series line chart reveals strong seasonality — sales consistently spike in November and December (holiday season) and dip in January–February. The 3-month moving average smooths the volatility and confirms a steady upward trend across all four years.

Key Insight: Q4 (October–December) is the revenue engine — the business should ensure inventory readiness and marketing spend peak during this window.

6.3 Profit Margin by Sub-Category

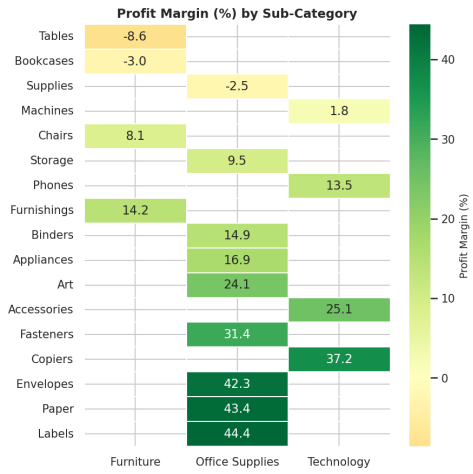

Figure 8: Profit Margin Heatmap by Sub-Category and Category

The heatmap instantly reveals the profitability landscape across all 17 sub-categories. Labels (44.4%), Paper (43.4%), and Envelopes (42.3%) have the highest margins, while Tables (–8.6%), Bookcases (–3.0%), and Supplies (–2.5%) are actively losing money. Copiers stand out as the most profitable Technology sub-category at 37.2% margin.

Key Insight: Three sub-categories are loss-makers: Tables (–$17.7K), Bookcases (–$3.5K), and Supplies (–$1.2K). These should be reviewed for pricing, discount, and vendor cost issues.

6.4 Discount Impact on Profitability

Figure 9: Discount vs Profit Scatter Plot, Coloured by Category

This scatter plot is one of the most critical visualizations in the analysis. It shows a clear, negative relationship between discount level and profit. Transactions with no discount generated $320,988 in total profit, while discounted transactions collectively lost $34,579. Items discounted above 20% destroyed $135,364 in profit. Furniture is hit hardest at high discount levels.

Key Insight: Discounts above 20% are a net-negative proposition. The company should implement a discount cap policy, especially for Furniture, and shift promotional strategy toward volume-based incentives.

6.5 Geographic Profit Performance

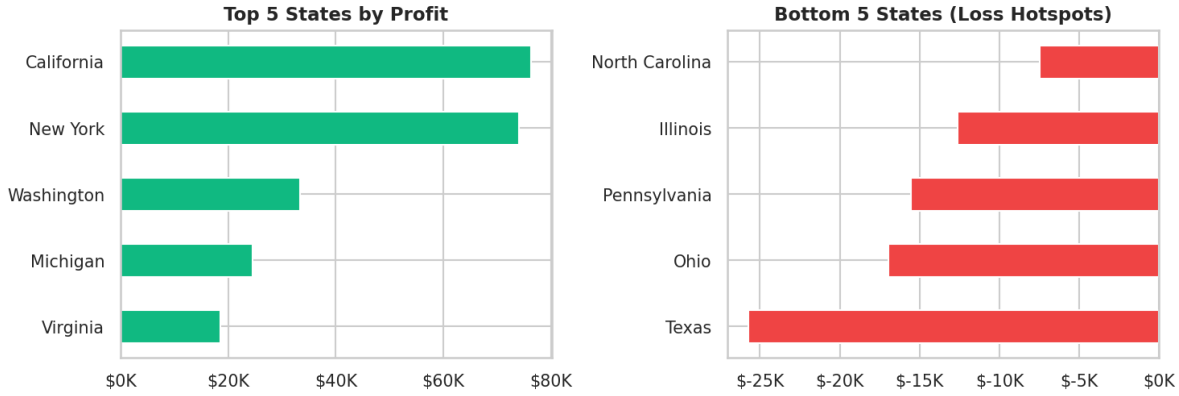

Figure 10: Top 5 and Bottom 5 States by Profit

California ($76K profit) and New York ($74K) dominate the profit landscape, accounting for over 50% of total profit together. On the loss side, Texas (–$25.7K), Ohio (–$16.5K), and Pennsylvania (–$15.6K) are significant drags, all predominantly in the Central region.

Key Insight: The Central region has the weakest margin (7.9% vs 14.9% for West). The loss-making states should be investigated for regional discount patterns, shipping cost issues, or product mix problems.

6.6 Cumulative Sales by Year (YTD Overlay)

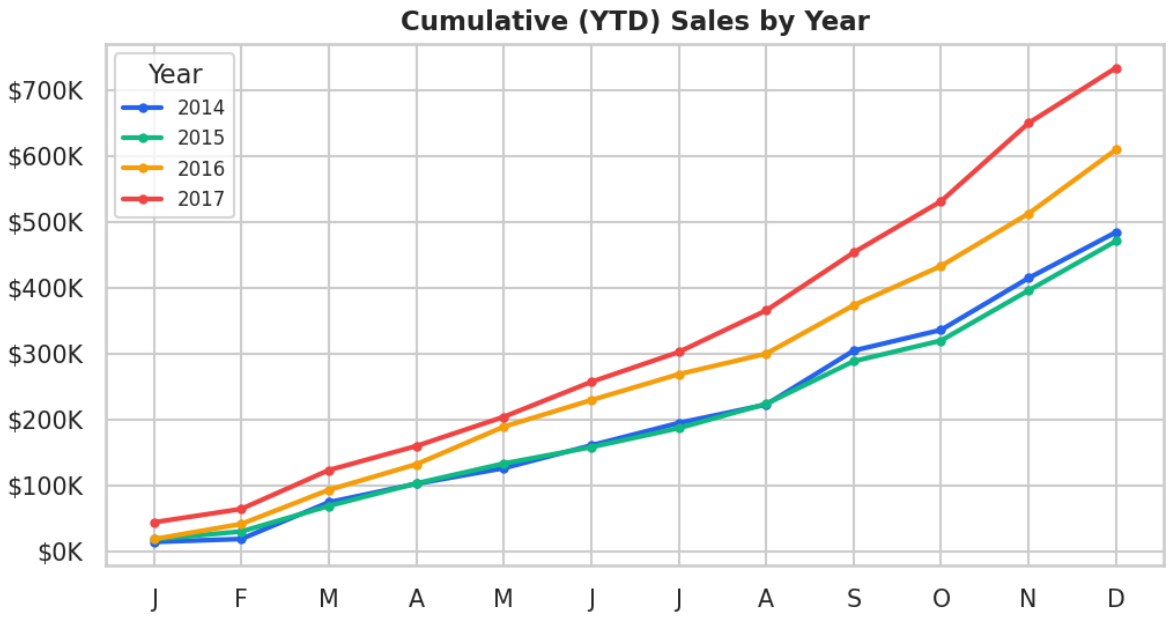

Figure 11: Year-to-Date Cumulative Sales — One Line Per Year

This overlay chart confirms that each subsequent year outperformed the previous one at virtually every month. The curves accelerate sharply in Q4, showing consistent holiday-season patterns. The gap between years widens in the second half, indicating that the business's growth was concentrated in H2.

Key Insight: Growth is real and accelerating, but heavily weighted toward H2. Strategies to boost H1 performance (Q1–Q2) could smooth revenue and reduce operational strain.

7. Key Insights & Findings

#	Finding	Impact
1	Sales grew 51% over 4 years ($484K → $733K)	Strong top-line growth trajectory; business is scaling
2	Profit margin peaked at 13.4% (2016), dipped to 12.7% (2017)	Growth came at cost of slight margin erosion — likely discount-driven
3	Tables lost $17.7K, Bookcases lost $3.5K, Supplies lost $1.2K	Three sub-categories actively destroying profit
4	No-discount items: +$321K profit. Discounted items: –$35K profit	Discounting is a net-negative — current strategy is broken
5	Discounts >20% destroyed $135K in profit	Clear threshold beyond which discounts are value-destructive
6	Central region margin: 7.9% (vs 14.9% West, 13.5% East)	Central region is the weakest performer — needs investigation
7	Texas, Ohio, Pennsylvania are the top loss-making states	Concentrated losses in Central — likely correlated with discount practices
8	98% of customers are repeat buyers (2+ orders)	Strong retention; focus should be on growing share of wallet
9	Consumer segment drives ~50% of sales	Largest segment but may be most price-sensitive
10	Q4 drives disproportionate revenue (Nov–Dec spike every year)	Holiday seasonality is reliable and should be operationally exploited

Figure 12: Summary of Key Findings

8. Recommendations

8.1 Discount Strategy Overhaul

The data clearly shows that discounts above 20% destroy profit. The company should implement a tiered discount cap: maximum 15% for Furniture (the most discount-sensitive category), 20% for Office Supplies, and 25% for Technology (which has the highest base margins to absorb discounts). Volume-based incentives should replace percentage-off promotions where possible.

8.2 Product Portfolio Review

Tables, Bookcases, and Supplies should undergo a formal review. Options include renegotiating vendor costs, increasing prices, reducing discount availability on these items, or — in extreme cases — discontinuing the worst-performing SKUs. Conversely, Copiers (37.2% margin) and Labels/Paper (43–44% margin) are strong performers that should receive increased marketing support.

8.3 Central Region Investigation

The Central region's 7.9% margin is roughly half of the West's 14.9%. A deep-dive into Central's discount patterns, product mix, and shipping costs is needed. The loss-making states (Texas, Ohio, Pennsylvania) should be audited individually to identify whether the issue is over-discounting, product mix, or structural cost.

8.4 Seasonal Inventory & Marketing Alignment

Given the consistent Q4 spike, the business should ensure inventory levels and marketing budgets are front-loaded for October–December. Conversely, Q1 represents an opportunity — targeted promotions (within the discount cap) could smooth the revenue curve and reduce the operational strain of extreme Q4 peaks.

8.5 Customer Retention & Growth

With 98% repeat buyers, retention is strong. The next lever is growing share of wallet — cross-selling high-margin products (Copiers, Accessories, Paper) to existing customers, and using the RFM segmentation from the SQL analysis to identify at-risk customers for proactive outreach.

9. Conclusion & Future Work

This project demonstrates a complete, production-grade data analytics pipeline — from raw CSV ingestion through data cleaning, relational database design, analytical SQL, and data visualization. The analysis surfaced several high-impact findings, most critically the destructive effect of heavy discounting and the underperformance of the Central region and Furniture category.

Technical Skills Demonstrated

- **Python:** Pandas for data cleaning, Matplotlib/Seaborn for visualization, Jupyter for documentation.
- **SQL:** Star schema design, ETL with staging tables, window functions (NTILE, LAG, RANK, running SUM), CTEs, FILTER clause.
- **Data Engineering:** Normalized schema design, indexing strategy, ON CONFLICT handling, post-load validation.
- **Business Analysis:** RFM segmentation, discount impact analysis, geographic profitability mapping, seasonality identification.

Future Work

- Build an interactive Power BI or Tableau dashboard for stakeholder self-service.
- Add predictive analytics — sales forecasting using time-series models (ARIMA, Prophet).
- Implement customer churn prediction using the RFM scores as features.
- Automate the pipeline with scheduled ETL (e.g., Airflow or dbt).
- Expand geographic analysis with a choropleth map visualisation.

This report was prepared as part of a data analytics portfolio project. All code, SQL scripts, and notebooks are available on GitHub.